DS605: Fundamentals of Machine Learning

Lab Assignment - 5

Machine Learning with Scikit-learn and From Scratch

Dataset: UCI Productivity Prediction of Garment Employees

Submission: Public GitHub repository link only.

Objective

Build regression and classification models with Scikit-learn, recreate the same workflow manually using only NumPy and Pandas, and compare predictive performance, execution time, and implementation efficiency.

Dataset and Targets

- Use the UCI Garment Employee Productivity dataset containing real production features such as department, team, overtime, incentives, work in progress, number of workers, targeted productivity, and actual productivity.
- Regression target: predict actual_productivity using Linear Regression.
- Classification target: create MeetsTarget = 1 when actual_productivity >= targeted_productivity, otherwise 0, and use Logistic Regression. Do not use actual_productivity as an input for classification.

Part A - Scikit-learn Implementation

- Prepare the dataset using suitable Scikit-learn preprocessing for missing values, categorical features, and scaling. Use a single fixed train-test split for all subsequent comparisons.
- Train LinearRegression and LogisticRegression. Record training and prediction time. Evaluate regression using MAE, RMSE, and R2; evaluate classification using accuracy, precision, recall, and F1-score.

Part B - From-Scratch Implementation

- Rebuild the same end-to-end workflow using only NumPy and Pandas. Scikit-learn and other machine-learning libraries are not allowed in this part.
- Implement missing-value handling, categorical encoding, feature scaling, Linear Regression, Logistic Regression, prediction, and all required evaluation metrics manually.
- Linear Regression may use a closed-form or iterative solution. Logistic Regression should include the sigmoid function, probability prediction, thresholding, and parameter optimization.
- Reuse the same train-test samples from Part A and use vectorized NumPy operations wherever possible.

Part C - Comparison and Optimization

- Create compact comparison tables for Scikit-learn and manual implementations using the same predictive metrics, training time, and prediction time.
- Improve the manual implementation using better preprocessing, vectorization, feature selection, learning-rate or convergence tuning, regularization, or another justified method.
- Try to match or improve the Scikit-learn scores and reduce the runtime gap. Beating Scikit-learn execution time is optional; explain why the final performance or runtime differs.

Expected Workflow

Raw Data -> Preprocessing -> Train-Test Split -> Model Training -> Prediction -> Evaluation -> Comparison -> Optimization

Important Requirement

- Do not call Scikit-learn preprocessing, model, metric, or train-test utilities inside the manual section. Python timing utilities may be used only to measure execution time.
- The final comparison should be fair and reproducible: use the same data split, features, target definitions, and evaluation metrics wherever applicable.

Deliverables (GitHub Only)

- Public GitHub repository containing the runnable Scikit-learn implementation, complete NumPy/Pandas implementation from scratch, comparison tables, timing results, optimized manual version, and README with key observations.
- Submit only the public GitHub repository link.